# 백트래킹

Backtracking

네 가지 코딩 문제로 배우는 선택과 되돌리기

# 기말 공개 문제는 네 개입니다

공개 24문제 중 기말에 10문제를 출제합니다.

| 번호 | 문제 | 난도 |
|---|---|---|
| 7-1 | 카드를 순서 있게 고르는 경우의 수 | 아주 쉬움 |
| 7-2 | 합이 목표인 부분집합의 개수 | 쉬움 |
| 7-3 | 이웃한 정점을 다른 색으로 칠하기 | 기본 |
| 7-4 | N-Queen 배치의 수 | 조금 도전 |

이 네 문제의 조건과 함수 규격을 기준으로 연습합니다.

# 백트래킹의 기본

선택하고, 확인하고, 되돌립니다.

## 선택

다음 후보 하나를 시도합니다.

## 조건 확인

조건을 어긴 가지는 더 진행하지 않습니다.

## 복구

다른 후보를 보려면 방금 선택을 되돌립니다.

# DFS와 가지치기

깊이 우선 탐색에 조건 검사를 붙여 불필요한 가지를 줄입니다.

| 개념 | 역할 |
| --- | --- |
| DFS | 한 가지를 끝까지 내려간 뒤 돌아오는 탐색 순서 |
| 가지치기 | 현재 조건으로 답이 될 수 없는 가지를 중단 |
| 백트래킹 | 선택을 시도하고 복구하며 가능한 답을 탐색 |

가지치기가 있어도 최악에는 많은 후보를 볼 수 있습니다.

# 수업 흐름

약 2시간 동안 네 문제를 직접 다룹니다.

0–20분: 카드 경우의 수와 선택 나무
20–45분: 부분집합 합
45–55분: 쉬는 시간
55–80분: 그래프 색칠
80–105분: N-Queen
105–120분: 한 문제 직접 구현·테스트

# 7-1 카드를 순서 있게 고르는 경우의 수

아주 쉬움

1부터 n까지 적힌 서로 다른 카드 n장 중 r장을 중복 없이 골라 일렬로 놓습니다. 순서가 다르면 다른 경우입니다. $0 \le r \le n \le 8$이며 r=0이면 아무것도 놓지 않는 1가지입니다. 가능한 경우의 수를 반환하세요.

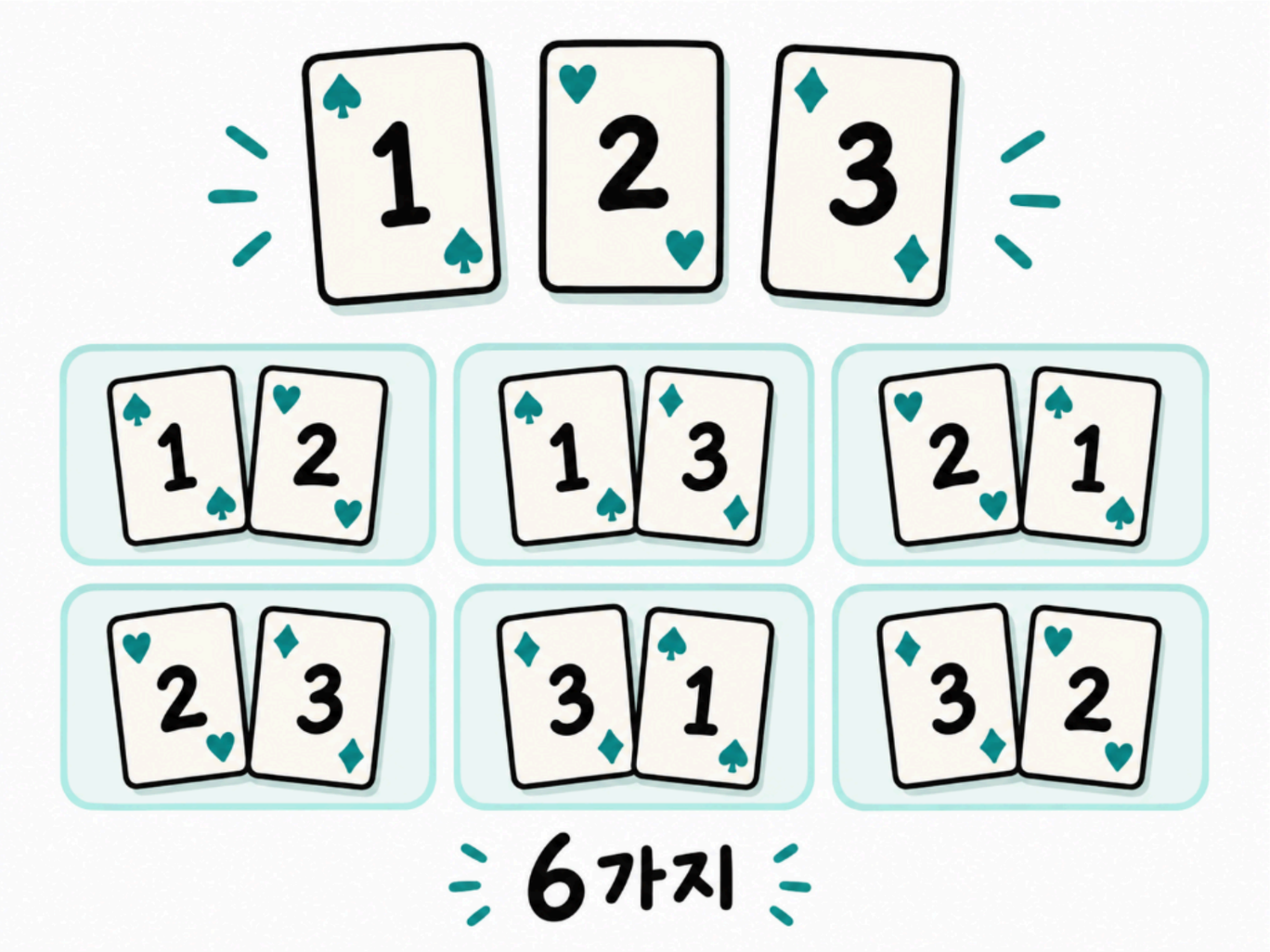
# 함수와 예제

Python: permutation_count(n, r)

| 입력 | 반환값 |
| --- | --- |
| 3, 2 | 6 |
| 4, 1 | 4 |
| 5, 0 | 1 |

```
int permutation_count(int n, int r) {
```

# 카드를 순서 있게 고르는 경우의 수의 작은 예




카드 3장 중 2장
순서를 구분하면
3×2=6가지입니다.

# 작은 입력을 손으로 추적합니다

코드를 읽기 전에 어떤 선택이 가능한지 확인합니다.

| 첫 카드 | 가능한 둘째 카드 | 경우 수 |
|---|---|---|
| 1 | 2, 3 | 2 |
| 2 | 1, 3 | 2 |
| 3 | 1, 2 | 2 |

첫 자리에는 n장, 다음 자리에는 n−1장 중 하나를 놓습니다. r자리까지 가능한 수를 곱합니다.

# 이 문제의 계산 방법

7-1

첫 자리에는 n장, 다음 자리에는 n-1장 중 하나를 놓습니다. r자리까지 가능한 수를 곱합니다.

r=0이면 빈 배치 한 가지이므로 1입니다. 카드의 순서를 구분하고 같은 카드는 반복하지 않습니다.

# 카드를 순서 있게 고르는 경우의 수 · Python

```
def permutation_count(n, r):
    answer = 1
    for i in range(r):
        answer *= n-i
    return answer
```

시간 O(r), 추가 공간 O(1)

# 카드를 순서 있게 고르는 경우의 수 · C

```
int permutation_count(int n, int r) {
    int answer=1;
    for (int i=0; i<r; ++i) answer*=n-i;
    return answer;
}
```

시간 O(r), 추가 공간 O(1)

# 경계 입력과 복잡도

7-1

## 경계 입력

r=0이면 빈 배치 한 가지이므로 1입니다. 카드의 순서를 구분하고 같은 카드는 반복하지 않습니다.

## 계산량

시간 O(r), 추가 공간 O(1)

# 직접 구현하고 다시 실행합니다

7-1 카드를 순서 있게 고르는 경우의 수

① 함수 이름과 입력 범위를 적습니다.
② 빈 코드에서 직접 구현합니다.
③ 공개 예제와 경계 입력을 실행합니다.
④ 실패한 입력을 고치고 다시 실행합니다.

# 7-2 합이 목표인 부분집합의 개수

쉬움

양의 정수 배열 a에서 몇 개의 원소를 골라 합이 target이 되는 선택의 수를 구하세요. 각 인덱스는 한 번만 고르며, 값이 같아도 인덱스가 다르면 다른 원소입니다. 길이는 0..20, 각 값은 1..50, target은 0..1000입니다. 아무것도 고르지 않는 선택도 포함합니다.

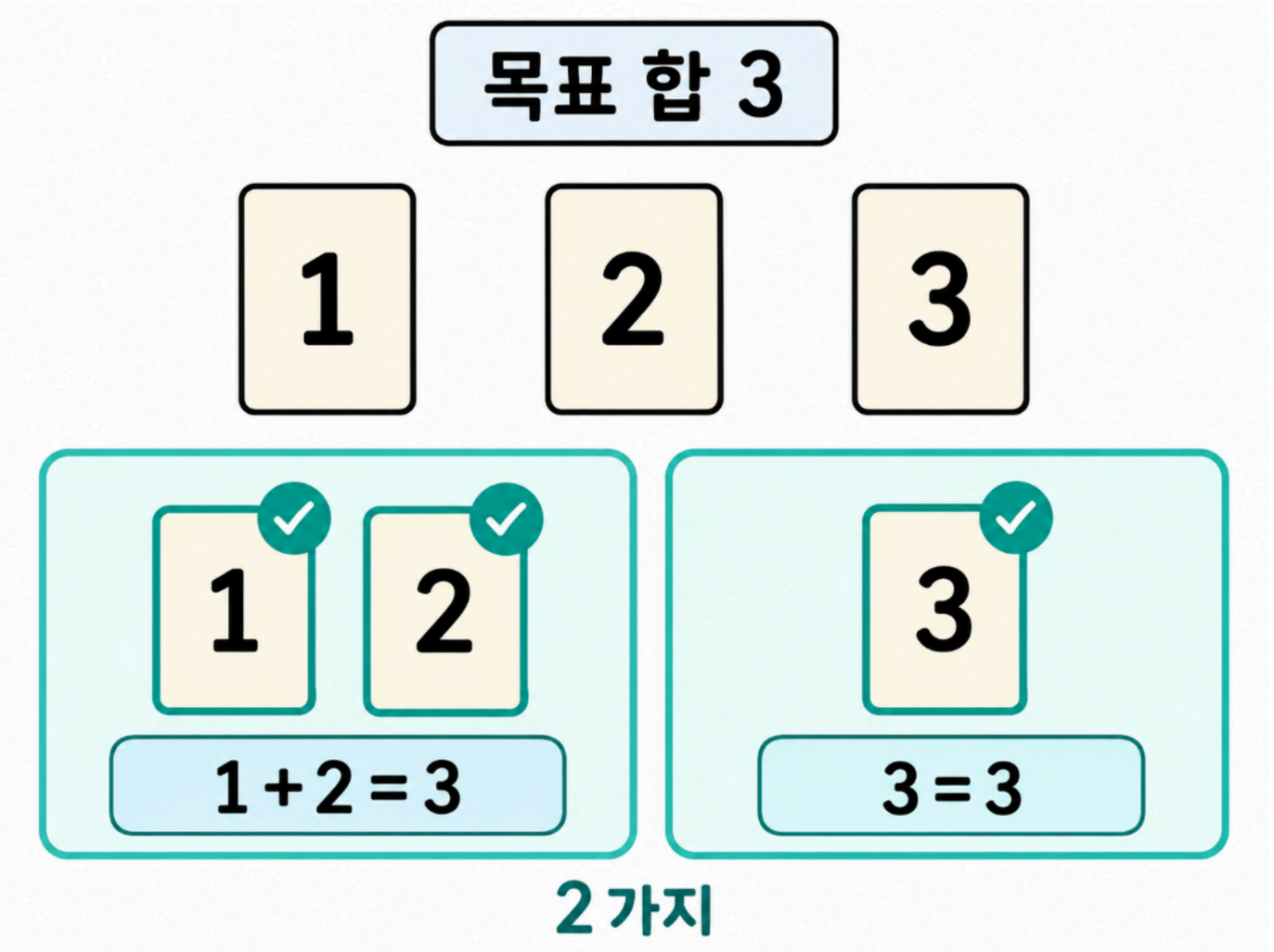
# 함수와 예제

Python: subset_sum_count(a, target)

| 입력 | 반환값 |
| --- | --- |
| [1,2,3], 3 | 2 |
| [1,1], 1 | 2 |
| [], 0 | 1 |

```
int subset_sum_count(const int a[], int n, int target) {
```

# 합이 목표인 부분집합의 개수의 작은 예




목표 합은 3
1과 2를 고르거나
3만 고릅니다.
답은 2가지입니다.

# 작은 입력을 손으로 추적합니다

코드를 읽기 전에 어떤 선택이 가능한지 확인합니다.

| 고른 인덱스의 값 | 합 | 목표 3과 같나? |
|---|---|---|
| 아무것도 없음 | 0 | 아니오 |
| 1과 2 | 3 | 예 |
| 3 하나 | 3 | 예 |
| 1과 3 | 4 | 아니오 |

각 원소를 안 고르는 가지와 고르는 가지로 나눕니다. 배열 끝에서 합이 target인지 확인해
경우의 수를 더합니다.

# 이 문제의 계산 방법

7-2

각 원소를 안 고르는 가지와 고르는 가지로 나눕니다. 배열 끝에서 합이 target인지 확인해 경우의 수를 더합니다.

모든 원소가 양수이므로 현재 합이 target을 넘으면 더 진행해도 줄어들지 않습니다. target=0에는 빈 선택 한 가지가 있습니다.

# 합이 목표인 부분집합의 개수 · Python

```
def subset_sum_count(a, target):
    def search(i, total):
        if total > target:
            return 0
        if i == len(a):
            return int(total == target)
        return (search(i+1, total)
                + search(i+1, total+a[i]))
    return search(0, 0)
```

최악 시간 $O(2^n)$, 재귀 스택 $O(n)$

# 합이 목표인 부분집합의 개수 · C

```
static int subset_search(const int a[], int n, int target,
                         int i, int total) {
    if (total>target) return 0;
    if (i==n) return total==target;
    return subset_search(a,n,target,i+1,total)
         + subset_search(a,n,target,i+1,total+a[i]);
}
int subset_sum_count(const int a[], int n, int target) {
    return subset_search(a,n,target,0,0);
}
```

최악 시간 $O(2^n)$, 재귀 스택 $O(n)$

# 경계 입력과 복잡도

7-2

## 경계 입력

모든 원소가 양수이므로 현재 합이 target을 넘으면 더 진행해도 줄어들지 않습니다.
target=0에는 빈 선택 한 가지가 있습니다.

## 계산량

최악 시간 $O(2^n)$, 재귀 스택 $O(n)$

# 직접 구현하고 다시 실행합니다

7-2 합이 목표인 부분집합의 개수

① 함수 이름과 입력 범위를 적습니다.
② 빈 코드에서 직접 구현합니다.
③ 공개 예제와 경계 입력을 실행합니다.
④ 실패한 입력을 고치고 다시 실행합니다.

# 10분 쉬었다가 이어갑니다

선택과 복구를 정리합니다.

다음은 정점의 색과 퀸의 위치를
조건에 맞게 배치하는 문제입니다.

# 7-3 이웃한 정점을 다른 색으로 칠하기

기본

정점 1..8개의 무방향 그래프가 0/1 인접 행렬 graph로 주어집니다.
graph[i][j]=1이면 두 정점이 이웃하며, 대칭이고 대각선은 0입니다.
1≤k≤3가지 색으로 모든 정점을 칠하되 이웃한 두 정점의 색이 달라야 합니다. 가능하면 1, 불가능하면 0을 반환하세요.

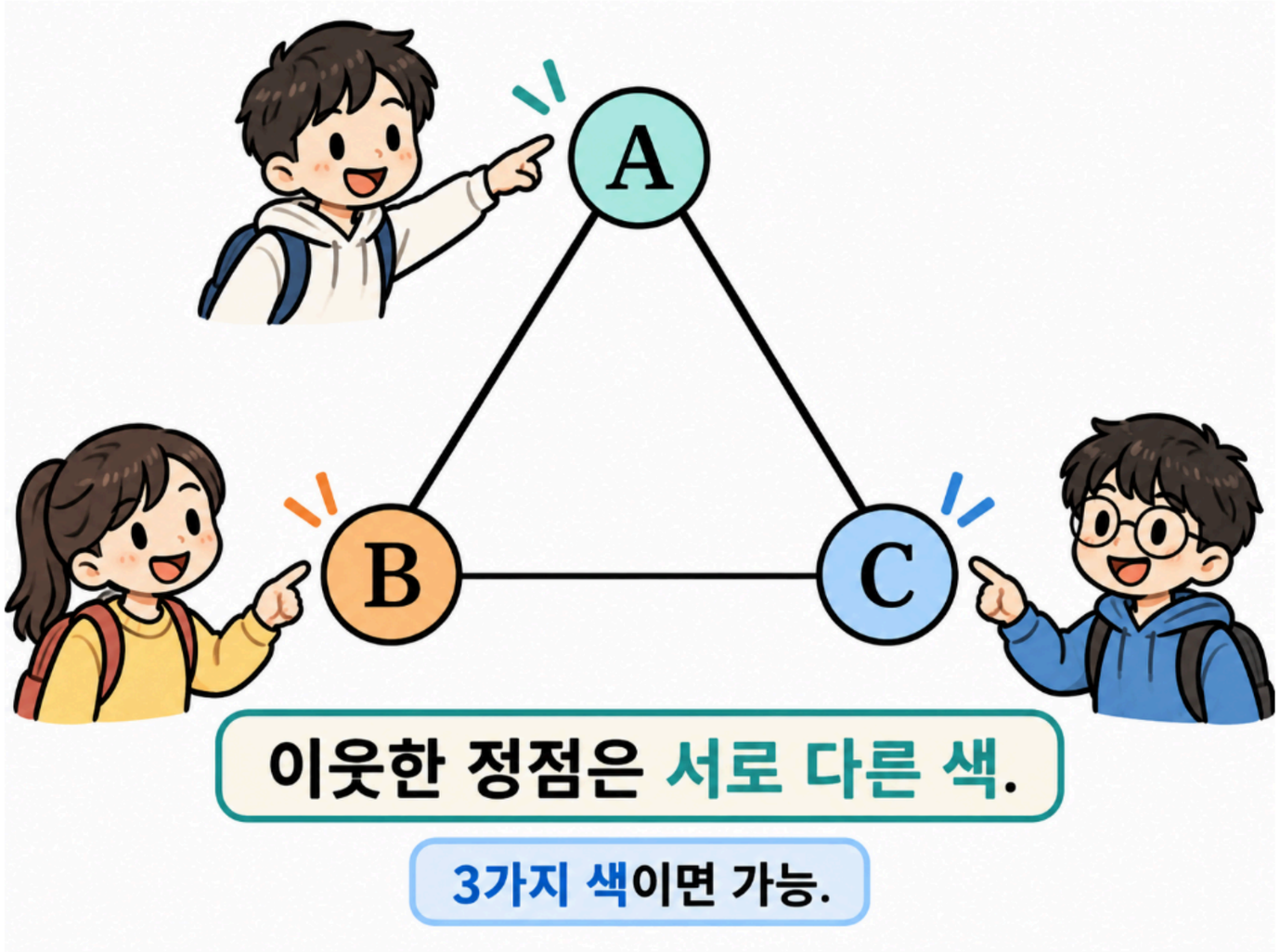
# 함수와 예제

Python: can_color(graph, k)

| 입력 | 반환값 |
| --- | --- |
| [[0,1,1],[1,0,1],[1,1,0]], 2 | 0 |
| [[0,1,1],[1,0,1],[1,1,0]], 3 | 1 |
| [[0]], 1 | 1 |

```
int can_color(const int graph[][8], int n, int k) {
```

# 이웃한 정점을 다른 색으로 칠하기의 작은 예




삼각형의 세 정점은
모두 이웃합니다.
세 색이면 가능,
두 색이면 불가합니다.

# 작은 입력을 손으로 추적합니다

코드를 읽기 전에 어떤 선택이 가능한지 확인합니다.

| 삼각형의 정점 | 선택할 색 | 판단 |
|---|---|---|
| A | 첫 번째 색 | 가능 |
| B | 두 번째 색 | A와 달라야 함 |
| C | 세 번째 색 | A·B와 모두 달라야 함 |

정점을 하나씩 칠합니다. 이미 칠한 이웃과 색이 같은 후보는 바로 제외하고, 뒤에서 막히면 현재 색을 지우고 다음 색을 시도합니다.

# 이 문제의 계산 방법

7-3

정점을 하나씩 칠합니다. 이미 칠한 이웃과 색이 같은 후보는 바로
제외하고, 뒤에서 막히면 현재 색을 지우고 다음 색을 시도합니다.

간선이 없는 그래프는 한 색으로 가능합니다. 삼각형은 두 색으로
불가능하고 세 색으로 가능합니다. 불가능할 때도 반드시 0을 반환합니다.

# 이웃한 정점을 다른 색으로 칠하기 · Python

```
def can_color(graph, k):
    color = [0]*len(graph)
    def paint(v):
        if v == len(graph):
            return 1
        for c in range(1, k+1):
            if all(not graph[v][u] or color[u] != c
                   for u in range(v)):
                color[v] = c
                if paint(v+1):
                    return 1
                color[v] = 0
        return 0
    return paint(0)
```

보수적 최악 시간 $O(n^2 \cdot k^n)$, 색 배열·재귀 스택 $O(n)$

# 이웃한 정점을 다른 색으로 칠하기 · C

```
static int paint(const int g[][8], int n, int k,
                 int v, int color[]) {
    if (v==n) return 1;
    for (int c=1; c<=k; ++c) {
        int ok=1;
        for (int u=0; u<v; ++u)
            if (g[v][u] && color[u]==c) ok=0;
        if (ok) { color[v]=c;
            if (paint(g,n,k,v+1,color)) return 1;
            color[v]=0; }
    } return 0;
}
int can_color(const int graph[][8], int n, int k) {
    int color[8]={0}; return paint(graph,n,k,0,color);
}
```

보수적 최악 시간 $O(n^2 \cdot k^n)$, 색 배열·재귀 스택 $O(n)$

# 경계 입력과 복잡도

7-3

## 경계 입력

간선이 없는 그래프는 한 색으로 가능합니다. 삼각형은 두 색으로 불가능하고 세 색으로 가능합니다. 불가능할 때도 반드시 0을 반환합니다.

## 계산량

보수적 최악 시간 $O(n^2 \cdot k^n)$, 색 배열·재귀 스택 $O(n)$

# 직접 구현하고 다시 실행합니다

7-3 이웃한 정점을 다른 색으로 칠하기

① 함수 이름과 입력 범위를 적습니다.
② 빈 코드에서 직접 구현합니다.
③ 공개 예제와 경계 입력을 실행합니다.
④ 실패한 입력을 고치고 다시 실행합니다.

# 7-4 N-Queen 배치의 수

조금 도전

n×n 체스판에 퀸 n개를 놓되 같은 행, 같은 열, 같은 대각선에 두 퀸이 없어야 합니다. 1≤n≤8입니다. 회전하거나 뒤집은 배치도 놓인 칸이 다르면 다른 배치로 세며, 가능한 배치의 수를 반환하세요.

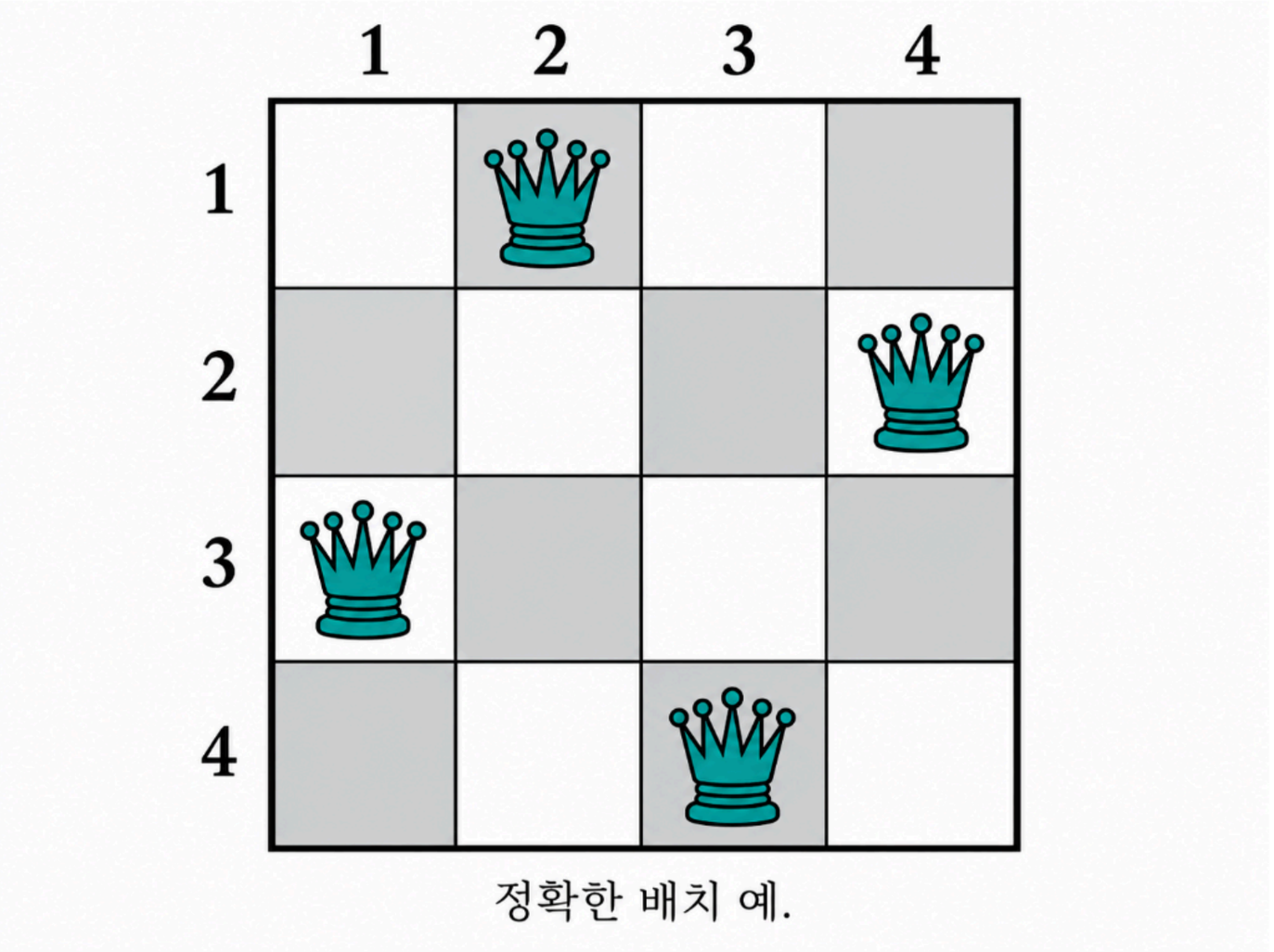
# 함수와 예제

Python: nqueen_count(n)

| 입력 | 반환값 |
| --- | --- |
| 1 | 1 |
| 2 | 0 |
| 4 | 2 |

```
int nqueen_count(int n) {
```

# N-Queen 배치의 수의 작은 예




정확한 배치 예.

4×4 체스판
열은 2, 4, 1, 3
서로 같은 열과
대각선이 없습니다.

# 작은 입력을 손으로 추적합니다

코드를 읽기 전에 어떤 선택이 가능한지 확인합니다.

| 행 | 퀸의 열 | 확인 |
|---|---|---|
| 1 | 2 | 다른 퀸 없음 |
| 2 | 4 | 열·대각선 충돌 없음 |
| 3 | 1 | 열·대각선 충돌 없음 |
| 4 | 3 | 열·대각선 충돌 없음 |

한 행에 하나씩 퀸을 놓습니다. 같은 열이거나 행 차이와 열 차이의 절댓값이 같은 후보는 대각선 충돌이므로 제외합니다.

# 이 문제의 계산 방법

7-4

한 행에 하나씩 퀸을 놓습니다. 같은 열이거나 행 차이와 열 차이의
절댓값이 같은 후보는 대각선 충돌이므로 제외합니다.

n=2와 n=3에는 배치가 없습니다. 대칭인 배치를 합쳐 세지 않습니다. n=4의
답은 2입니다.

# N-Queen 배치의 수 · Python

```
def nqueen_count(n):
    columns = []
    def place(row):
        if row == n:
            return 1
        answer = 0
        for col in range(n):
            if all(col != c and abs(col-c) != row-r
                   for r, c in enumerate(columns)):
                columns.append(col)
                answer += place(row+1)
                columns.pop()
        return answer
    return place(0)
```

열 중복을 배제하며 탐색. 충돌 확인을 포함한 보수적 시간 $O(n^2 \cdot n!)$, 공간 $O(n)$

# N-Queen 배치의 수 · C

```
static int queens(int n, int row, int pos[]) {
    if (row==n) return 1;
    int answer=0;
    for (int col=0; col<n; ++col) {
        int ok=1;
        for (int r=0; r<row; ++r) {
            int d=col-pos[r]; if (d<0) d=-d;
            if (col==pos[r] || d==row-r) ok=0;
        }
        if (ok) {pos[row]=col; answer+=queens(n,row+1,pos);}
    } return answer;
}
int nqueen_count(int n) {
    int pos[8]={0}; return queens(n,0,pos);
}
```

열 중복을 배제하며 탐색. 충돌 확인을 포함한 보수적 시간 $O(n^2 \cdot n!)$, 공간 $O(n)$

# 경계 입력과 복잡도

7-4

## 경계 입력

n=2와 n=3에는 배치가 없습니다. 대칭인 배치를 합쳐 세지 않습니다. n=4의 답은 2입니다.

## 계산량

열 중복을 배제하며 탐색. 충돌 확인을 포함한 보수적 시간 $O(n^2 \cdot n!)$, 공간 $O(n)$

# 직접 구현하고 다시 실행합니다

7-4 N-Queen 배치의 수

① 함수 이름과 입력 범위를 적습니다.
② 빈 코드에서 직접 구현합니다.
③ 공개 예제와 경계 입력을 실행합니다.
④ 실패한 입력을 고치고 다시 실행합니다.

# 7장의 네 문제 정리

공개 문제의 범위를 다시 확인합니다.

| 문제 | 반환값 | 핵심 |
|---|---|---|
| 7-1 | 경우의 수 | 카드를 순서 있게 고르는 경우의 수 |
| 7-2 | 경우의 수 | 합이 목표인 부분집합의 개수 |
| 7-3 | 가능 1 / 불가능 0 | 이웃한 정점을 다른 색으로 칠하기 |
| 7-4 | 경우의 수 | N-Queen 배치의 수 |

조건을 읽고, 가능한 선택을 빠짐없이 계산합니다.

# 기말 준비

공개 24문제 중 10문제

공개된 함수 규격과 입력 범위를 공부합니다.
별도 객관식·서술형 없이 코딩테스트로 평가합니다.
7장에서는 7-1부터 7-4까지 네 문제를 연습합니다.

# 선택을 되돌려 다음 가능성을 봅니다

Backtracking

7-1 카드 순서
7-2 부분집합 합
7-3 그래프 색칠
7-4 N-Queen